

Revisão

1) Criando o projeto com Vite.

Resposta: Projeto criado com React + TypeScript utilizando Vite e execução do servidor local.

```
npm create vite@latest app -- --template react-ts
cd app
npm install
npm run dev
```

2) Criando a Store com Redux Toolkit.

Resposta: Instalação do Redux Toolkit, criação da store com configureStore e utilização do Provider envolvendo o App.

```
npm install @reduxjs/toolkit react-redux

const store = configureStore({
  reducer: {}
})

<Provider store={store}>
  <App/>
</Provider>
```

3) Criar slice de usuários.

Resposta: Criação de slice com estado inicial e reducers addUser e removeUser.

```
const userSlice = createSlice({
  name: 'users',
  initialState: {users: []},
  reducers: {
    addUser: (state, action) => {
      state.users.push(action.payload)
    },
    removeUser: (state, action) => {
      state.users = state.users.filter(
        u => u !== action.payload
      )
    }
  }
})
```

4) Listar usuários no componente.

Resposta: Utilização do useSelector para leitura do estado e botão disparando addUser.

```
const users = useSelector((state) => state.users.users)
const dispatch = useDispatch()

<ul>
  {users.map((u, i) => (
```

```

    <li key={i}>{u}</li>
  )}
</ul>

<button onClick={()=>dispatch(addUser("Pedro"))}>
Adicionar
</button>

```

5) Criar slice de hábitos.

Resposta: Criação de reducers addHabit, removeHabit e filterHabits.

```

const habitSlice = createSlice({
  name: 'habits',
  initialState: { habits: [], filtered: [] },
  reducers: {
    addHabit: (state, action) => {
      state.habits.push(action.payload)
    },
    removeHabit: (state, action) => {
      state.habits = state.habits.filter(
        h => h.id !== action.payload
      )
    },
    filterHabits: (state, action) => {
      state.filtered = state.habits.filter(
        h => h.category === action.payload
      )
    }
  }
})

```

6) Formulário de cadastro de hábitos.

Resposta: Inputs controlados com useState e disparo do reducer addHabit.

```

const [nome, setNome] = useState("")
const [categoria, setCategoria] = useState("")

const enviar = () => {
  dispatch(addHabit({
    id: Date.now(),
    nome,
    categoria
  })))
}

```

7) Implementar filtro de hábitos.

Resposta: Select de categorias disparando filterHabits.

```

<select onChange={e =>
  dispatch(filterHabits(e.target.value))
}>
<option>Saúde</option>
<option>Estudo</option>
<option>Lazer</option>
</select>

```

8) Criar hook personalizado.

Resposta: Hook encapsulando useSelector e useDispatch.

```
export const useUsers = ()=>{
  const dispatch = useDispatch()
  const users = useSelector(
    state=>state.users.users
  )

  const add = (u)=>dispatch(addUser(u))
  const remove = (u)=>dispatch(removeUser(u))

  return {users,add,remove}
}
```

9) Montar mini dashboard.

Resposta: Separação em dois componentes: gerenciamento de usuários e gerenciamento de hábitos.

10) Integração API Alpha Vantage.

Resposta: B

11) Classificação IMC.

Resposta: Condição retornando cor e classificação.

```
if(imc >= 35 && imc <= 39.9){
  return {
    cor:'red',
    classificacao:'Obesidade Grau II'
  }
}
```

12) Controle de loading.

Resposta: Ativar loading no início e desativar após sucesso ou erro.

```
setLoading(true)

try{
  const res = await axios.get(url)
}catch{
}finally{
  setLoading(false)
}
```

13) Dependências useEffect.

Resposta: C

14) Configuração tsconfig target ES2015.

Resposta: Permite uso de recursos modernos como async/await; ES5 pode gerar incompatibilidades.

15) Uso de slice(0,10).

Resposta: B

16) keyExtractor no FlatList.

Resposta: B

17) Evento Button React Native.

Resposta: Uso da propriedade onPress para execução da função.

```
<Button  
  title="Calcular"  
  onPress={calcular}  
/>
```

18) Validação de campo vazio.

Resposta: Verificação e exibição de mensagem em vermelho.

```
if(!nota){  
  setErro("Por gentileza, insira números válidos")  
}
```

```
<Text style={{color:'red'}}>  
{erro}  
</Text>
```

19) Axios vs Fetch.

Resposta: B

20) Propriedade value TextInput.

Resposta: Vincula o valor do campo ao estado, mantendo input controlado.

21) Renderização condicional média.

Resposta: Uso do operador lógico.

```
{calculado && media >= 6 && (  
<Text>Aprovado</Text>  
)}
```

22) useEffect com array vazio.

Resposta: Executa apenas uma vez; com variável executa ao alterar.

23) Flexbox React Native.

Resposta: Centralização com justifyContent e alignItems.

24) Tratamento de erro com axios.

Resposta: Atualização do estado de erro melhora feedback ao usuário.

25) Propriedade value TextInput.

Resposta: B

26) Variáveis dinâmicas URL axios.

Resposta: B

```
axios.get(`https://api.com/${ticker}`)
```

27) Validação isNaN.

Resposta: Evita cálculo com valores inválidos.

```
if(isNaN(n1) || isNaN(n2)){  
  return  
}
```

28) FlatList vs ScrollView.

Resposta: B

29) Uso de slice(0,5).

Resposta: D

30) Propriedade obrigatória FlatList.

Resposta: C

```
<FlatList
  data={dados}
  renderItem={renderItem}
/>
```